

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
CENTRO REGIONAL DE CHIRIQUI
LABORATORIO DE DESARROLLO DE MÓVILES

Nombre: Francisco Hernández

Cedula: 4-823-2336

Analiza las siguientes interrogantes y responde la de acuerdo a tus conocimientos.

1. ¿Qué ventajas ofrece el diseño adaptativo en comparación con el diseño fluido? En qué escenarios es preferible cada enfoque?

El diseño adaptativo crea diferentes versiones para diferentes pantallas por separado para una app, las cuales varían entre las columnas o filas que muestre según el tamaño, en cambio, en el diseño fluido, los elementos de la app se reorganizan según el tamaño de la pantalla. El adaptativo es mejor si se necesita cambiar mucho la interfaz entre dispositivos, es decir, entre una Tablet y una computadora, y en cuanto al diseño fluido sería mejor cuando una interfaz es más simple y solo necesita que el contenido se acomode un poco.

2. ¿Cómo funciona el widget Wrap y en qué situaciones sería útil?

Wrap es un widget que envuelve a sus hijos o componentes, parecido al Row o Column, pero lo bueno de este es que se cambia a la siguiente línea si no hay suficiente espacio, a diferencia de Row y Column. Es útil en situaciones donde se crean etiquetas o botones dinámicos donde la cantidad de elementos o el tamaño de la pantalla puede variar.

3. ¿Cuál es el propósito de un Theme en flutter y cómo se aplica a una aplicación?

Se utiliza para mantener un solo estilo visual en la aplicación y generalizarlo, es decir que se pueda aplicar en cualquier parte de la aplicación. Se aplica definiendo un objeto ThemeData a la propiedad theme del widget principal MaterialApp.

4. Explica cómo se utiliza la clase TextTheme para definir y aplicar estilos de texto en una aplicación Flutter?

Esta clase agrupa diferentes estilos de texto y se define dentro de ThemeData general de la aplicación, esto le asigna propiedades como el tamaño de fuente, peso, color a cada categoría del texto.

5. Describe algunas técnicas avanzadas de estilo que se pueden utilizar para mejorar la apariencia de una aplicación Flutter.

- Animaciones Implícitas y Explícitas: Usar widgets como `AnimatedContainer` o `AnimationController` para transiciones fluidas.
- Glassmorphism: Utilizar `BackdropFilter` con un efecto de desenfoque (blur) sobre contenedores semi-transparentes para lograr un efecto de vidrio esmerilado.
- CustomPainters: Dibujar formas, curvas o gráficos vectoriales completamente personalizados píxel por píxel usando la clase `CustomPainter`.
- ThemeExtensions: Crear propiedades de diseño personalizadas (design tokens) que no existen en el `ThemeData` estándar de Material Design.

6. Cómo puedes detectar la plataforma en la que se ejecuta tu aplicación Flutter ?

Se puede detectar importando la librería `dart:io` y utilizando la clase `Platform`. `Platform.isAndroid` o `Platform.isIOS`, para Android y para iOS respectivamente.

7. Cómo puedes personalizar los estilos de tu aplicación Flutter ?

Se pueden personalizar globalmente utilizando `ThemeData` en el `MaterialApp`, importando fuentes de Google Fonts o locales en el `pubspec.yaml`, también se puede personalizar creando widgets reutilizables que tengan el estilo que queramos.

8. Cuándo debes usar `setState()` en Flutter ?

Se usa `setState()` para gestionar el estado de un único `StatefulWidget`, es decir, cuando hay un cambio en la interfaz gráfica que solo afecta a ese widget específico y no necesita ser compartido con el resto de la aplicación.

9. En qué situaciones serían más apropiados usar `ListView` en lugar de `GridView`.

- `ListView` es apropiado usarlo cuando se necesita mostrar elementos secuenciales en una lista unidimensional (ya sea vertical u horizontal). Ejemplos: un feed de noticias, una lista de chats o un menú de configuraciones.
- `GridView` es mejor cuando necesitamos organizar los elementos en dos dimensiones (filas y columnas). Ejemplos: una galería de fotos, un catálogo de productos o un tablero de ajedrez.

10. Investiga y compara brevemente Provider, Riverpod y Bloc.

- **Provider:** Es una envoltura sobre InheritedWidget. Es fácil de aprender, excelente para inyección de dependencias y proyectos medianos, pero depende estrictamente del árbol de widgets, lo que puede causar excepciones si se busca un Provider fuera de contexto.
- **Riverpod:** Creado por el mismo autor de Provider para solucionar sus fallas. Es seguro en tiempo de compilación (evita errores de Provider no encontrado), no depende del árbol de widgets y es altamente escalable.
- **Bloc (Business Logic Component):** Utiliza un enfoque basado en eventos y Streams. Separa de manera estricta y predecible la lógica de negocio de la interfaz. Es robusto para aplicaciones empresariales grandes y complejas, pero tiene una curva de aprendizaje más pronunciada y requiere escribir más código repetitivo (boilerplate).

11. Cuáles son las mejores prácticas para manejar errores al consumir APIs

- Envolver las llamadas asíncronas en bloques try-catch.
- Validar siempre los códigos de estado HTTP (ej. 200 para éxito, 400/500 para errores) antes de procesar los datos.
- Crear clases de excepciones personalizadas para diferentes tipos de errores
- Mostrar mensajes de error amigables al usuario mediante la interfaz (ej. SnackBars) en lugar de exponer los errores crudos del servidor o del código.

12. Cómo se puede mapear datos JSON a clases de modelo en Flutter ?

Se crean clases en Dart que representan la estructura del JSON. Dentro de la clase, se define un constructor de fábrica (factory constructor), típicamente llamado `fromJson(Map<String, dynamic> json)`, que toma el mapa de datos decodificado y asigna los valores a las propiedades de la clase. Para proyectos profesionales, es una excelente práctica usar herramientas de generación de código como `json_serializable` o `freezed` para automatizar este proceso.

13. Qué es FutureBuilder y cómo se usa para mostrar datos asíncronos ?

`FutureBuilder` es un widget que construye la interfaz basándose en el estado de una operación asíncrona (`Future`). Utiliza sus propiedades `future` y `builder` para evaluar el progreso de la tarea en tiempo real mediante un objeto `AsyncSnapshot`, lo que le permite renderizar automáticamente un indicador de carga, los datos resueltos o un mensaje de error según corresponda, agilizando el desarrollo al no requerir la gestión manual del estado con `setState`.

14. Cuáles son las limitaciones de setState en aplicaciones flutter complejas

- Reconstrucciones masivas: Llamar a setState reconstruye todo el árbol de widgets debajo de donde se llamó. Si no se aísla bien, puede afectar el rendimiento repintando partes de la UI que no cambiaron.
- Dificultad para compartir estado: Si dos widgets muy separados en el árbol necesitan la misma información, pasar variables y callbacks hacia arriba y abajo (prop drilling) usando setState se vuelve inmanejable.
- Acoplamiento: Mezcla la lógica de negocio con la interfaz gráfica, lo que dificulta mantener y probar el código.

15. Cómo funciona la arquitectura Provider?

La arquitectura Provider es un patrón de gestión de estado e inyección de dependencias que actúa como un envoltorio optimizado de InheritedWidget. Funciona centralizando la información en una clase de estado (como ChangeNotifier) a la que se suscriben los widgets descendientes; cuando los datos cambian, se invoca el método notifyListeners(), desencadenando la reconstrucción exclusiva de los componentes suscritos para optimizar el rendimiento de la aplicación y evitar repintados innecesarios.